Estructura de Datos

Tema 6:

Listas

GRADO EN INGENIERÍA DEL SOFTWARE

Salvador Sánchez y Cristian Rodríguez

Universidad
Rey Juan Carlos

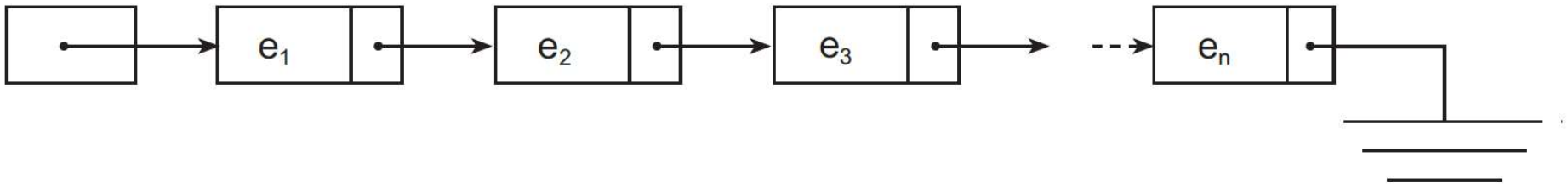
Curso 2023-2024

Contenidos

- Definiciones
- El TAD Lista
- Implementación
  - Lista enlazada simple
  - Lista circular
  - Lista doblemente enlazada
- Listas y genericidad
- Lista como base de implementación de otras estructuras de datos
- Ejercicios propuestos

Lista: definiciones

- Estructura de datos lineal, dinámica, que representa una secuencia de elementos, a menudo del mismo tipo.
- Colección o secuencia de elementos dispuestos uno detrás de otro, en la que cada elemento se conecta al siguiente elemento por un “enlace”.



TAD Lista

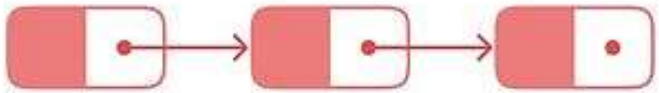
- Por su flexibilidad, no existe una única definición del TAD lista: debe determinarse primero el tipo de lista que se desea, lo cual condiciona sus operaciones (contrato)
  - Por ejemplo: una lista sin restricciones permite operaciones no permitidas en una lista ordenada tales como `insertar_final`
- En todo caso, todas comparten operaciones básicas:
  - Consultar si está vacía (`is_empty`)
  - Inicializar/crear

TAD Lista

- Una abstracción extrema de una lista incluiría las siguientes operaciones (su implementación será diferente en función del tipo de lista):
  - Meter un elemento (`insert`)
  - Eliminar un elemento (`delete`)
  - Consultar si está vacía (`is_empty`)
  - Conocer el primer y último elemento (`first`/`last`)
  - Comprobar si un elemento está (`in`)
  - \+ Inicializar/crear

TAD Lista: especificación

- **Nombre:** Lista.
- **Valores posibles:** *vacía, otros (según elementos)*.
- **Operaciones:**
  - `insert_at_end`
  - `insert_at_begin`
  - `delete_at_end`
  - `delete_at_begin`
  - `first`
  - `last`
  - `is_empty`
  - `in_list`
- **Uso de otros tipos:** tipo base, punteros



Implementación y tipos

- La implementación de listas en estructuras de soporte estáticas como arrays no es habitual: Siempre en memoria dinámica
- Tipos (por implementación):
  - Lista enlazada
  - Lista doblemente enlazada
  - Lista circular
- Otros tipos (por lógica interna): Lista ordenada

Lista enlazada simple

Implementación con memoria dinámica

- Utilizamos una estructura nodo

```
type
    nodo = record
        info: integer;
        sig: ^nodo;
    end;
```

- Y luego una estructura con los dos manejadores de la lista:

```
    tlista = record
        first, last: ^nodo;
    end;
```

Lista enlazada: implementación

```
{ Operaciones básicas }
procedure initialize(var list: tlista);
function is_empty(list: tlista): boolean;
function first(list: tlista): integer;
function last(list: tlista): integer;
procedure insert_at_end(var list: tlista; x: integer);
procedure insert_at_begin(var list: tlista; x: integer);
procedure delete_at_end(var list: tlista);
procedure delete_at_begin(var list: tlista);
procedure delete(var list: tlista; x:integer);
function in_list(list: tlista; x:integer): boolean;
function rec_in_list(list:tlista; x:integer): boolean;

{ Otras operaciones }
procedure show(list: tlista);
procedure clear(var list: tlista);
function num_elems(list: tlista): integer;
procedure copy(list: tlista; var c2: tlista);
```

Lista enlazada: implementación

```
implementation

  procedure initialize(var list: tlista);
  begin
    list.first := nil;
    list.last := nil;
  end;

  function is_empty(list: tlista): boolean;
  begin
    is_empty:= (list.first = nil) and (list.last = nil);
  end;

  function first(list: tlista): integer;
  begin
    first:= list.first^.info;
  end;

  function last(list: tlista): integer;
  begin
    last:= list.last^.info;
  end;
```

Lista enlazada: implementación

```
procedure insert_at_end(var list: tlista; x: integer);
var
  new_node: ^nodo;
begin
  new(new_node);
  new_node^.info:= x;
  new_node^.sig:= nil;
  if is_empty(list) then
   begin
    list.first:=new_node;
    list.last:= new_node;
   end
  else
   begin
    list.last^.sig:=new_node;
    list.last:= new_node;
   end;
end;
```

Lista enlazada: implementación

```
procedure insert_at_begin(var list: tlista; x: integer);
var
  new_node: ^nodo;
begin
  new(new_node);
  new_node^.info:= x;
  new_node^.sig:= list.first;
  if is_empty(list) then
    list.last:= new_node;
  list.first:=new_node;
end;
```

Lista enlazada: implementación

```
procedure delete_at_begin(var list: tlista);
var
  aux: ^nodo;
begin
  if not is_empty(list) then
  begin
    aux:= list.first;
    list.first := list.first^.sig;
    if list.first = nil then list.last:= nil;
    dispose(aux);
  end;
end;

procedure delete_at_end(var list: tlista);
var
  act, ant: ^nodo;
begin
  { Muy ineficiente: implica recorrer la lista -> O(n) }
end;
```

Lista enlazada: Interfaz

```
{ Operaciones básicas }
procedure initialize(var list: tlista);
function is_empty(list: tlista): boolean;
function first(list: tlista): integer;
function last(list: tlista): integer;
procedure insert_at_end(var list: tlista; x: integer);
procedure insert_at_begin(var list: tlista; x: integer);
procedure delete_at_end(var list: tlista);
procedure delete_at_begin(var list: tlista);
procedure delete(var list: tlista; x:integer);
function in_list(list: tlista; x:integer): boolean;
function rec_in_list(list:tlista; x:integer): boolean;
```

Lista enlazada: implementación

```
function in_list(list: tlista; x:integer): boolean;
var
  encontrado: boolean;
  aux:^nodo;
begin
  encontrado:= false;
  if not is_empty(list) then
    begin
      aux:= list.first;
      while (not encontrado) and (aux<>nil) do
      begin
        encontrado:= x = aux^.info;
        aux:= aux^.sig;
      end;
    end;
  in_list:= encontrado;
end;
```

Lista enlazada: implementación

```
function rec_in_list(list:tlista; x:integer): boolean;
{ Versión recursiva de "in" }
var
  resto: tlista;
begin
  if is_empty(list) then rec_in_list:=false
  else
    begin
      resto.first:= list.first^.sig;
      if list.first^.sig = nil then resto.last:=nil
      else resto.last:= list.last;
      rec_in_list := (x=list.first^.info) or (rec_in_list(resto,x));
    end;
end;
```

Lista enlazada: implementación

```
procedure show(list: tlista);
var
  aux: ^nodo;
begin
  if is_empty(list) then writeln('Empty list: No elements to show')
  else
  begin
    aux:= list.first;
    while aux<>nil do
    begin
      write(aux^.info,'-');
      aux:= aux^.sig
    end
  end;
  writeln;
end;

procedure clear(var list: tlista);
begin
  while not is_empty(list) do delete_at_begin(list);
end;
```

Lista enlazada: implementación

- Versión recursiva de show:

```
procedure rec_show(list: tlista);
var
  resto: tLista;
begin
  if not is_empty(list) then
  begin
    resto.first:= list.first^.next;
    if resto.first=nil then resto.last:=  nil
    else resto.last:=  list.last;
    write(list.first^.info,'-');
    rec_show(resto);
  end
end;
```

Lista enlazada: implementación

```
function num_elems(list: tlista): integer;
var
  aux: ^nodo;
  count: integer;
begin
  count:= 0;
  aux:= list.first;
  while aux<>nil do
  begin
      count:=count + 1;
      aux:= aux^.sig
  end;
  num_elems:= count
end;
```

Lista enlazada: implementación

- Versión recursiva de num_elems:

```
function rec_num_elems(list: tlista): integer;
var
  resto: tlista;
begin
  if is_empty(list) then rec_num_elems:= 0
  else
    begin
      resto.first:= list.first^.next;
      if resto.first=nil then resto.last:=  nil
      else resto.last:=  list.last;
      rec_num_elems:= 1 + rec_num_elems(resto);
    end;
end;
```

Lista enlazada: implementación

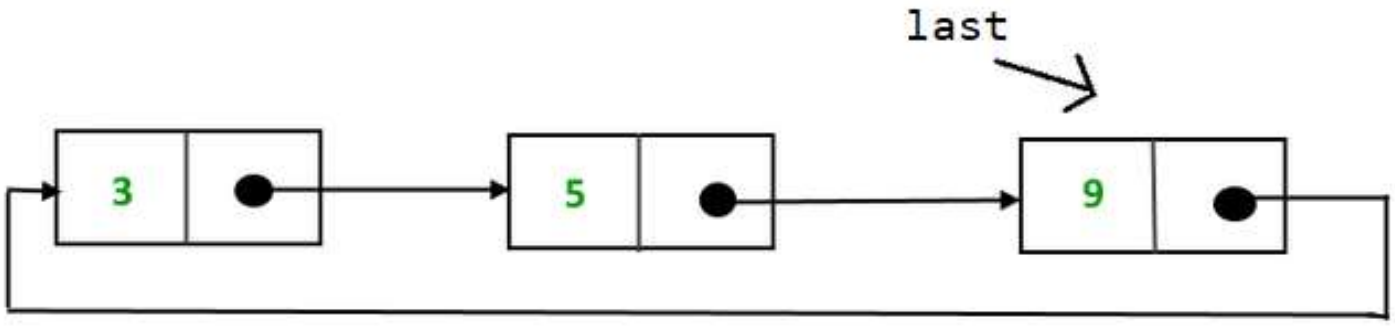
```
procedure copy(list: tlista; var c2: tlista);
var
  aux: ^nodo;
begin
  initialize(c2);
  if not is_empty(list) then
  begin
    aux:=list.first;
    while aux<>nil do
    begin
      insert_at_end(c2,aux^.info);
      aux:= aux^.sig
    end;
  end
end;
```

Lista circular

Lista circular

- El último nodo contiene un puntero al primero, de modo que:
  - La lista circular no tiene principio ni fin.
  - No hay ningún NIL en la parte siguiente de ninguno de los nodos.

- Solo un manejador, apuntando al final
- El primero se consigue como el siguiente del último



Lista circular

- La estructura de manejo es un único puntero y el nodo tiene la misma estructura que en una lista simple:

```
type
    nodo = record
        info: integer;
        sig: ^nodo;
    end;
```

- Y luego un manejador de la lista:

```
tListaCircular = ^nodo;
```

Lista circular

- Sus operaciones pueden variar según el uso que se le quiera dar
- Normalmente da igual guardar el orden de ingreso, por lo que tras una operación de inserción o borrado el puntero manejador podría cambiar

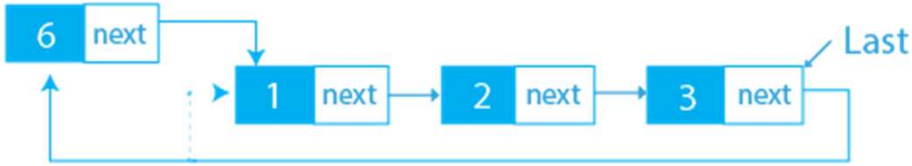
```
{ Operaciones básicas }
procedure initialize(var list: tListaCircularEnteros);
function is_empty(list: tListaCircularEnteros): boolean;
function first(list: tListaCircularEnteros): integer;
function last(list: tListaCircularEnteros): integer;
procedure insert_at_end(var list: tListaCircularEnteros; x: integer);
procedure insert_at_begin(var list: tListaCircularEnteros; x: integer);
procedure delete(var list: tListaCircularEnteros; x: integer);
function in_list(list: tListaCircularEnteros; x:integer): boolean;

{ Otras operaciones }
procedure clear(var list: tListaCircularEnteros);
procedure show(list: tListaCircularEnteros);
function num_elems(list: tListaCircularEnteros): integer;
procedure copy(list: tListaCircularEnteros; var c2: tListaCircularEnteros);
procedure delete_at_end(var list: tListaCircularEnteros);
procedure delete_at_begin(var list: tListaCircularEnteros);
```

Lista circular

```
procedure initialize(var list: tListaCircularEnteros);
begin
  list := nil;
end;

function is_empty(list: tListaCircularEnteros): boolean;
begin
  is_empty:= (list = nil);
end;

function first(list: tListaCircularEnteros): integer;
{ PRE: not empty }
begin
  first:= list^.sig^.info;
end;

function last(list: tListaCircularEnteros): integer;
{ PRE: not empty }
begin
  last:= list^.info;
end;
```

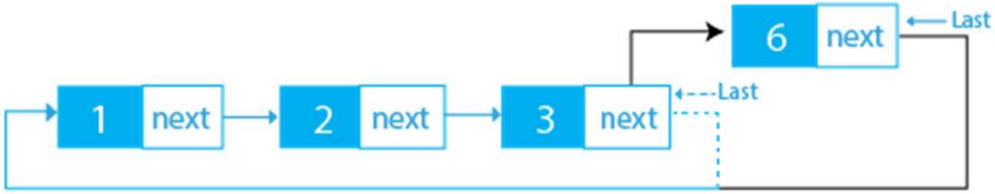
Lista circular

```
procedure insert_at_begin(var list: tListaCircularEnteros; x: integer);
var
  new_node: ^nodo;
begin
  new(new_node);
  new_node^.info:= x;
  if is_empty(list) then
   begin
     new_node^.sig:= new_node;
     list:= new_node;
   end
  else
   begin
     new_node^.sig:= list^.sig;
     list^.sig:=new_node;
   end
end;
```



Lista circular

```
procedure insert_at_end(var list: tListaCircularEnteros; x: integer);
var
  new_node: ^nodo;
begin
  new(new_node);
  new_node^.info:= x;
  if is_empty(list) then
   begin
     new_node^.sig:= new_node;
     list:= new_node;
   end
  else
   begin
     new_node^.sig:= list^.sig;
     list^.sig:=new_node;
     list:= new_node
   end;
end;
```



Lista circular

```
procedure delete(var list: tListaCircularEnteros; x: integer);
var
  start, aux: tListaCircularEnteros;
  encontrado: boolean;
begin
  if not is_empty(list) then
    if (list = list^.sig) and (list^.info = x) then
      begin
        dispose(list);
        list:= nil;
      end
    else
      begin
        start:= list;
        encontrado:= false;
        repeat
          aux:= list^.sig;
          if list^.sig^.info = x then
            begin
              list^.sig:= aux^.sig;
              dispose(aux);
              encontrado:= true
            end
          else
            list:= aux;
        until encontrado or (list=start);
      end;
    end;
```

Lista circular

```
function in_list(list: tListaCircularEnteros; x:integer): boolean;
var
  start: ^nodo;
  encontrado: boolean;
begin
  if is_empty(list) then in_list:= false
  else if list=list^.sig then in_list:= list^.info = x
  else
    begin
        start:= list;
        encontrado:= false;
        repeat
          if list^.sig^.info = x then
               encontrado:= true
          else
            list:= list^.sig;
        until encontrado or (list=start);
        in_list:= encontrado;
    end;
end;
```

Lista circular

```
procedure clear(var list: tListaCircularEnteros);
var
  aux: tListaCircularEnteros;
begin
  if not is_empty(list) then
    begin
      while list^.sig <> list do
        begin
           aux:= list^.sig;
           list^.sig:= aux^.sig;
           dispose(aux)
        end;
      dispose(list);
      list:= nil
    end;
end;
```

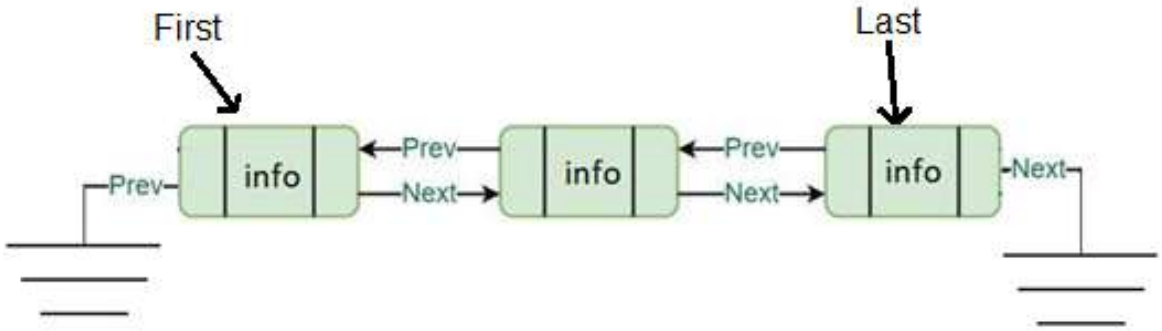
Lista circular

```
procedure show(list: tListaCircularEnteros);
var
  aux: ^nodo;
begin
  if is_empty(list) then writeln('Empty list: No elements to show')
  else
  begin
    aux:= list^.sig;
    repeat
      write(aux^.info,'-');
      aux:= aux^.sig;
    until aux=list^.sig;
  end;
  writeln;
end;
```

Lista doblemente enlazada

Lista doblemente enlazada

- Lista doblemente enlazada



- Ventajas:
  - Facilita inserciones y borrados en ambos extremos (y en general en cualquier lugar de la lista)
  - Simplifica recorridos en ambos sentidos
  - Ideal para la implementación de estructuras tales como grafos, diccionarios y otras

Lista doblemente enlazada

```
type
  nodo = record
    info: integer;
    next,prev: ^nodo;
  end;

  tlistaDoble = record
    first, last: ^nodo
  end;

{ Operaciones básicas }
procedure initialize(var list: tlistaDoble);
function is_empty(list: tlistaDoble): boolean;
function first(list: tlistaDoble): integer;
function last(list: tlistaDoble): integer;
procedure insert_at_end(var list: tlistaDoble; x: integer);
procedure insert_at_begin(var list: tlistaDoble; x: integer);
procedure delete_at_end(var list: tlistaDoble);
procedure delete_at_begin(var list: tlistaDoble);
function in_list(list: tlistaDoble; x:integer): boolean;

{ Otras operaciones }
procedure show(list: tlistaDoble);
procedure clear(var list: tlistaDoble);
function num_elems(list: tlistaDoble): integer;
procedure copy(list: tlistaDoble; var c2: tlistaDoble);
```

Lista doblemente enlazada

```
procedure initialize(var list: tlistaDoble);
begin
  list.first := nil;
  list.last := nil;
end;

function is_empty(list: tlistaDoble): boolean;
begin
  is_empty:= (list.first = nil) and (list.last = nil);
end;

function first(list: tlistaDoble): integer;
begin
  first:= list.first^.info;
end;

function last(list: tlistaDoble): integer;
begin
  last:= list.last^.info;
end;
```

Lista doblemente enlazada

```
procedure insert_at_end(var list: tlistaDoble; x: integer);
var
  new_node: ^nodo;
begin
  new(new_node);
  new_node^.info:= x;
  new_node^.next:= nil;
  new_node^.prev:= list.last;
  if is_empty(list) then
    list.first:=new_node
  else
    list.last^.next:=new_node;
  list.last:= new_node;
end;
```

Lista doblemente enlazada

```
procedure insert_at_begin(var list: tlistaDoble; x: integer);
var
  new_node: ^nodo;
begin
  new(new_node);
  new_node^.info:= x;
  new_node^.prev:= nil;
  new_node^.next:= list.first;
  if is_empty(list) then
    list.last:= new_node
  else
    list.first^.prev:=new_node;
  list.first:=new_node;
end;
```

Lista doblemente enlazada

```
procedure delete_at_begin(var list: tlistaDoble);
var
  aux: ^nodo;
begin
  if not is_empty(list) then
  begin
    aux:= list.first;
    list.first := list.first^.next;
    if list.first = nil then list.last:= nil
    else list.first^.prev:= nil;
    dispose(aux);
  end;
end;
```

Lista doblemente enlazada

```
procedure delete_at_end(var list: tlistaDoble);
var
  aux: ^nodo;
begin
  if not is_empty(list) then
  begin
    aux:= list.last;
    list.last := list.last^.prev;
    if list.last = nil then list.first:= nil
    else list.last^.next:= nil;
    dispose(aux);
  end;
end;
```

Lista doblemente enlazada

```
procedure show(list: tlistaDoble);
var
  aux: ^nodo;
begin
  if is_empty(list) then writeln('Empty list: No elements to show')
  else
  begin
    aux:= list.first;
    while aux<>nil do
    begin
      write(aux^.info,'-');
      aux:= aux^.next
    end
  end;
  writeln;
end;
```

Lista doblemente enlazada

```
function in_list(list: tlistaDoble; x:integer): boolean;
var
  encontrado: boolean;
  aux:^nodo;
begin
  if is_empty(list) then encontrado:= false
  else
  begin
    encontrado:= false;
    aux:= list.first;
    while (not encontrado) and (aux<>nil) do
    begin
      encontrado:= x = aux^.info;
      aux:= aux^.next;
    end;
  end;
  in_list:= encontrado;
end;
```

Lista doblemente enlazada

```
procedure clear(var list: tlistaDoble);
begin
  while not is_empty(list) do delete_at_begin(list);
end;

function num_elems(list: tlistaDoble): integer;
var
  aux: ^nodo;
  count: integer;
begin
  count:= 0;
  aux:= list.first;
  while aux<>nil do
  begin
      count:=count + 1;
      aux:= aux^.next
  end;
  num_elems:= count
end;
```

Lista doblemente enlazada

```
procedure copy(list: tlistaDoble; var c2: tlistaDoble);
var
  aux: ^nodo;
begin
  initialize(c2);
  if not is_empty(list) then
  begin
    aux:=list.first;
    while aux<>nil do
    begin
      insert_at_end(c2,aux^.info);
      aux:= aux^.next
    end;
  end
end;
```

Listas y genericidad

Limitaciones del enfoque anterior

- Hasta ahora hemos visto la aplicación de las listas a un ejemplo sencillo con ánimo de facilitar su comprensión: listas de enteros
- Al igual que ocurría con el resto de las estructuras vistas, las listas de otros elementos tienen idéntico comportamiento
- Programaremos listas genéricas en función de un elemento TElemento que igual que siempre debe implementar…
  - `assign(var e: TElemento; e2: TElemento);`
  - `lessThan(e: TElemento; e2: TElemento): boolean;`
  - `equalTo(e: TElemento; e2: TElemento): boolean;`
  - …

Unit Lista enlazada simple genérica

- Implementar unidad ListaGenerica en función de TElemento
  - … se propone como ejercicio ☺

Unit Lista enlazada doble genérica

- Implementar una unidad ListaDobleGenerica en función de TElemento
  - … se propone como ejercicio ☺

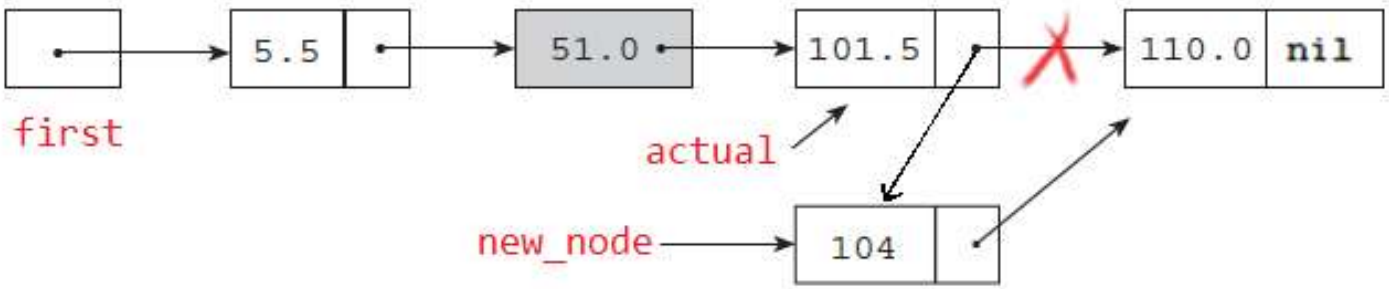
Listas como base para la implementación de otras estructuras

Base para estructuras más complejas

- La flexibilidad de las listas, tanto enlazadas como doblemente enlazadas, permite la implementación de estructuras con restricciones adicionales, o más complejas tales como:
  - Listas ordenadas
  - Colas de prioridades
  - Colas doblemente terminadas (*deques*)
  - Mapas / Diccionarios
  - Grafos

Lista ordenada

- Estructura de datos dinámica, cuyos elementos se encuentran en todo momento ordenados.
- La forma de insertar un elemento en una lista ordenada siempre realiza la operación de tal forma que la lista resultante mantiene la propiedad de ordenación
  - Para ello, en primer lugar, determina la posición de inserción y, a continuación, ajusta los enlaces.



Lista ordenada

```
procedure insert(var list: tSortedList; x: integer);
var
  new_node, actual: ^nodo;
begin
  new(new_node);
  new_node^.info:= x;
  if is_empty(list) or (x < list.first^.info) then // insertar el primero
    begin
      new_node^.next:= list.first;
      list.first:= new_node;
    end
  else
    begin  // Situar actual en el nodo anterior a la inserción
      actual:= list.first;
      while (actual^.next <> nil) and (x > actual^.next^.info) do
          actual:= actual^.next;
      new_node^.next:= actual^.next;
      actual^.next:= new_node;
    end;
end;
```

Cola de prioridad

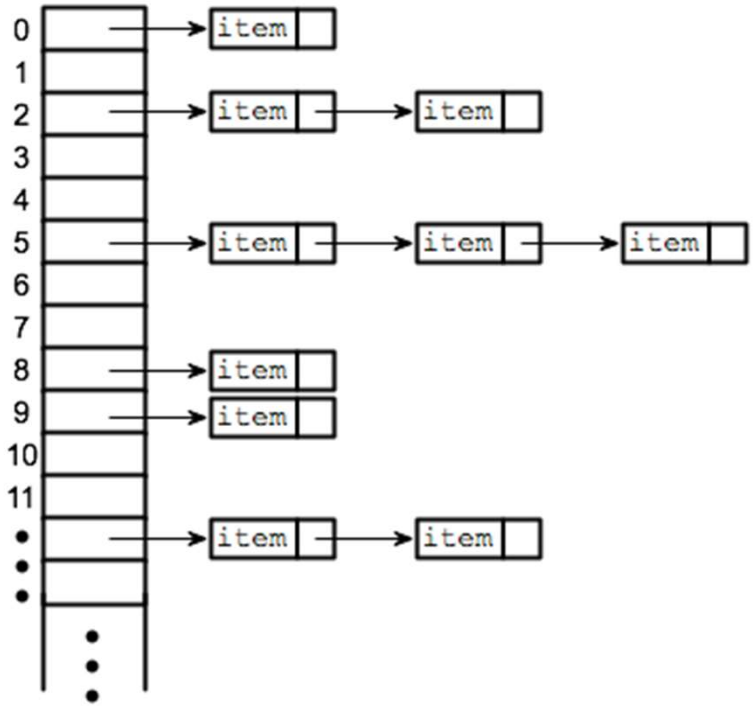
- Cada elemento de la cola lleva una prioridad asociada.
- Los componentes de la cola se ordenan según su prioridad.
- Hay 2 alternativas de implementación:
  - Añadir un campo a cada nodo con su prioridad, manteniendo la cola ordenada por un doble criterio: prioridad y a igual prioridad FIFO.
  - Crear tantas colas como prioridades haya, y almacenar cada elemento en su cola.
- Habitualmente si dos elementos tienen la misma prioridad se sirve primero al que lleva más tiempo esperando en la cola.

Cola doblemente terminada (*deque*)

- Una *deque* (pronunciado “DEK”) es una cola que permite inserciones en ambos extremos y no solo en el final.
- Útil en aplicaciones donde por algún motivo uno de los elementos debe entrar en un nivel prioritario (al inicio de la cola)
  - Un restaurante donde un cliente VIP “se cuela”
  - Cuando el elemento saliente no ha podido ser servido y se reinserta en la primera posición para no esperar de nuevo toda la cola.
  - etc.
- Una implementación obvia es una limitación de la lista general.

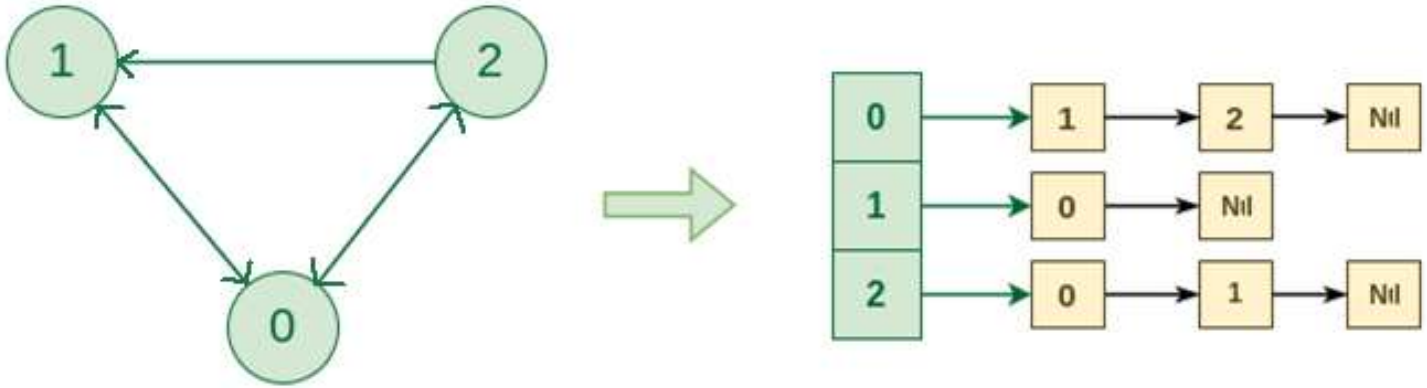
Diccionarios / Mapas

- Existe una lista de claves, cada una de las cuales almacena un enlace a una lista de valores asociados a dicha clave.
- La ordenación de las claves es crítica ya que es necesario hacer búsquedas constantemente por clave, tanto para inserción como para borrado.
  - Funciones Hash: HashMap
  - Lista ordenada de claves



Grafos

- Un grafo puede implementarse mediante listas de listas:
  - Una lista de nodos
  - Una lista de nodos adyacentes a un cierto nodo (listas de adyacencia)



(más sobre esto, pronto)

Ejercicios propuestos

Ejercicio propuesto 1

Calcula la complejidad en notación O grande de todas las operaciones de la lista enlazada simple vista en este tema.

```
{ Operaciones básicas }
procedure initialize(var list: tlista);
function is_empty(list: tlista): boolean;
function first(list: tlista): integer;
function last(list: tlista): integer;
procedure insert_at_end(var list: tlista; x: integer);
procedure insert_at_begin(var list: tlista; x: integer);
procedure delete_at_end(var list: tlista);
procedure delete_at_begin(var list: tlista);
function in_list(list: tlista; x:integer): boolean;
function rec_in_list(list:tlista; x:integer): boolean;

{ Otras operaciones }
procedure show(list: tlista);
procedure clear(var list: tlista);
function num_elems(list: tlista): integer;
procedure copy(list: tlista; var c2: tlista);
```

Ejercicio propuesto 2

Implementa el TAD `Deque` genérico en función de un elemento genérico `Telemento`.

```
{ Operaciones básicas }
procedure initialize(var d: tDeque);
procedure insert_last(var d: tDeque; x: TElemento);
procedure insert_first(var d: tDeque; x: TElemento);
procedure delete_first(var d: tDeque);
function first(d: tDeque): TElemento;

{ Otras operaciones }
function last(d: tDeque): TElemento;
function num_elems(var d: tDeque): integer;
function is_empty(d: tDeque): boolean;
```

Ejercicio propuesto 3

(a) Implementa una operación que permita invertir una lista enlazada simple

(b) Implementa la misma operación para una lista doblemente enlazada

No es necesario leer la entrada

Complejidad de tiempo esperada: O(n)

Espacio auxiliar esperado: O(1)

Ejercicio propuesto 4

Implementa mediante una lista ordenada las operaciones de inserción y servicio (pop) de una cola de procesos con prioridad (0-máxima prioridad, 3-minima).

Complejidad de tiempo esperada inserción: O(n)

Complejidad de tiempo esperada servicio (pop) : O(n)

Espacio auxiliar esperado: O(1)

Ejercicio propuesto 4bis

Implementa el sistema anterior mediante una cola multinivel de 4 colas.

Complejidad de tiempo esperada inserción: O(n)

Complejidad de tiempo esperada servicio (pop) : O(n)

Espacio auxiliar esperado: O(1)

Ejercicio propuesto 5

Implementa la operación de eliminación de un elemento por clave en una lista ordenada

No es necesario leer la entrada

Complejidad de tiempo esperada: O(n)

Espacio auxiliar esperado: O(1)

Ejercicio propuesto 6 (reto)

- A partir de un archivo de texto con tweets, genera un diccionario en el que la clave serán los hashtags y los valores serán los tweets que contengan dicho hashtag.
- Utiliza una lista ordenada para la lista de claves y calcula la complejidad de la operación de inserción.

Hashtag	tweets	
#kiwi	See you tonight Sydney City! Look for the wasted guy doing the haka. #Kiwi hahahaha.	Usual weekend of sports entertainment resumes in NZ on @skysportnz this wkend! #SuperRugby #NRL #NBL #ALeague #kiwias #kiwi #kiwiana
#NZMaori	Going off to see the #nzmaori game today. Probability be more expat kiwis at the game than locals.	In order to #NZMaori I will be swapping out my own Pakeha DNA with some spare Māori genes that Miriama Kamo has.
#thehaka	So I don't know anything about #Rugby but I do know #TheHaka; Kiwi yr7 teacher had us do it :D Manly rugby boys doing it's a better view tho	